

DHTOL Analyzer - Explication du projet pour développeurs juniors

1. Quel est le but de ce projet ?

Ce projet est un projet d'apprentissage pour automatiser plus tard l'analyse de campagnes de test DHTOL.

DHTOL signifie que des boards électroniques fonctionnent pendant une durée prévue sous stress. Le logiciel doit aider à répondre à des questions comme :

- quels boards faisaient partie du test ;
- dans quelle zone et à quelle position se trouvait chaque board ;
- si le test tournait en mode HV ou MV ;
- combien de temps le test était planifié ;
- combien de temps de stress chaque board a enregistré ;
- quelle version de firmware a été utilisée ;
- s'il existe un écart calculé entre la durée planifiée et la durée

journalisée.

Point important : le code documenté ici correspond au socle des semaines 1 et

2. Il sert de base propre pour les étapes suivantes : interface Streamlit, lecture TDMS, parsing des logs board, graphiques et analyse des défauts.

Le flux principal est :

```
Configuration JSON -> ParsedConfig + OvenplanEntry
Fichier MTPX       -> durée de test planifiée en secondes
Fichier DATA      -> BoardMetadata avec durée de stress + firmware
```

Le projet n'est donc pas seulement une interface. Il commence par des modèles de données fiables et des parseurs testables.

2. Structure des dossiers et fichiers

Les dossiers importants sont :

```
models/
  Modèles de données communs.

parsers/
  Fonctions qui lisent des fichiers et les transforment en données Python
  propres.

analysis/
  Logique d'analyse : défauts, glitches, temps de stress et attribution du
  courant.

visualization/
  Fonctions d'affichage et graphiques.

tests/
  Tests automatisés.

docs/
```

Documentation des décisions techniques.

Dans les premières semaines, les points centraux sont :

```
README.md
docs/week-01-foundation.md
docs/week-02-parsers.md
models/data_models.py
parsers/config_json.py
parsers/mtpx.py
parsers/board_data.py
tests/
```

Un exemple d'utilisation côté développeur :

```
from parsers.config_json import parse_config_json

parsed = parse_config_json("run_A_test.json")
```

3. Où commence le programme ?

Pour les utilisateurs finaux, l'interface peut être lancée via Streamlit quand l'application est prête :

```
python -m streamlit run app.py
```

Pour les développeurs, les points d'entrée techniques sont surtout les fonctions de parsing :

```
parse_config_json(path)
parse_planned_test_seconds(path)
parse_board_data(path)
```

Les tests se lancent avec :

```
pytest -q
```

La configuration `pytest.ini` indique à pytest où chercher les modules et les tests :

```
[pytest]
pythonpath = .
testpaths = tests
```

Pourquoi c'est utile ?

- `pythonpath = .` permet à Python d'importer `models`, `parsers`, `analysis` et `visualization` depuis le dossier du projet.
- `testpaths = tests` indique que les tests se trouvent dans `tests`.

Sans cette configuration, un débutant rencontre souvent une erreur comme :

```
ModuleNotFoundError: No module named 'parsers'
```

4. Flux de données

Le flux de données volontairement simple est :

```
Chemin de fichier
-> fonction de parsing
-> objet Python propre ou valeur simple
-> test, analyse ou visualisation
```

Exemple JSON :

```
run_A_test.json
-> parse_config_json()
-> ParsedConfig(...)
```

Exemple MTPX :

```
run.mtpx
-> parse_planned_test_seconds()
-> 3603600
```

Exemple DATA :

```
board.data
-> parse_board_data()
-> BoardMetadata(log_stress_seconds=123.5, firmware_version="9.0")
```

Pourquoi ce flux est bon :

- chaque type de fichier possède son propre parseur ;
- chaque parseur a une responsabilité claire ;
- les tests peuvent vérifier chaque responsabilité séparément ;
- l'analyse n'a pas besoin de connaître les structures JSON brutes.

Erreur fréquente chez les débutants : tout mettre dans une seule grande fonction. Le code devient alors difficile à tester, difficile à relire et difficile à corriger.

5. Fichier `models/data_models.py`

Ce fichier définit les formes de données communes. Les parseurs renvoient ces objets au lieu de renvoyer des dictionnaires libres.

Imports

```
from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
from pathlib import Path
from typing import Optional
```

Rôle des imports :

- `from __future__ import annotations` rend les type hints plus souples ;
- `dataclass` crée des classes de données légères ;
- `field` sert aux valeurs par défaut sûres, par exemple les listes ;
- `Enum` définit des valeurs autorisées fixes ;
- `Path` représente les chemins de fichiers ;
- `Optional` signifie qu'une valeur peut être présente ou valoir `None`.

Pourquoi utiliser des objets ?

Un dictionnaire libre est rapide à écrire :

```
result = {"zone": "A", "position": 1}
```

Mais une faute de frappe arrive vite :

```
result["postion"]
```

Avec des dataclasses, la structure est claire et plus facile à tester.

Classe Zone

```
class Zone(Enum):  
    A = "A"  
    B = "B"  
    C = "C"
```

Cette classe définit les zones autorisées du four.

Pourquoi elle existe :

- le système connaît les zones A, B et C ;
- le code ne doit pas stocker des chaînes arbitraires ;
- `Zone.A` est plus clair que `"A"`.

Erreur fréquente :

```
zone = "a"  
zone = "Zone A"  
zone = " A "
```

Pour un humain ces valeurs se ressemblent. Pour Python, ce sont des valeurs différentes.

Classe TempMode

```
class TempMode(Enum):  
    HV = "HVoltage"  
    MV = "MVoltage"
```

Cette classe décrit si le test tourne en mode HV ou MV.

Pourquoi pas seulement un booléen ?

```
is_mv = True
```

`TempMode.MV` explique le domaine métier directement. `True` demande au lecteur de se souvenir de ce que le booléen signifie.

Dataclass OvenplanEntry

```
@dataclass  
class OvenplanEntry:  
    controller_id: Optional[int]  
    position: int  
    zone: Zone  
    dut_name: str  
    hw_target: str  
    load_board: str = ""  
    dut_board: str = ""  
    uc_fsm: str = ""
```

Cette classe décrit une ligne de l'ovenplan JSON.

Champs importants :

- `controller_id` : numéro extrait d'un nom DUT comme 88_1_2 ;
- `position` : slot lu dans le champ JSON `Slot` ;
- `zone` : zone A, B ou C ;
- `dut_name` : nom DUT original ;
- `hw_target` : hardware target ;
- `load_board`, `dut_board`, `uc_fsm` : informations complémentaires.

Connaissance métier importante :

`controller_id != position`

Exemple :

```
Nom DUT      : 88_1_2
Controller ID : 88
Slot         : 1
```

Le contrôleur 88 ne signifie pas "position 88". Dans l'exemple, il est à la position 1.

Dataclass `ParsedConfig`

```
@dataclass
class ParsedConfig:
    test_name: str
    zone: Optional[Zone]
    temp_mode: TempMode
    instruments: list[str] = field(default_factory=list)
    ovenplan_entries: list[OvenplanEntry] = field(default_factory=list)
    source_path: Optional[Path] = None
    warnings: list[str] = field(default_factory=list)
```

Cette classe décrit le résultat du parseur JSON.

Pourquoi `field(default_factory=list)` ?

Les listes sont modifiables. Cette écriture crée une nouvelle liste pour chaque objet.

À éviter :

```
warnings: list[str] = []
```

À utiliser :

```
warnings: list[str] = field(default_factory=list)
```

Sinon, plusieurs objets pourraient partager la même liste.

Dataclass `BoardMetadata`

```
@dataclass
class BoardMetadata:
    log_stress_seconds: float = 0.0
    firmware_version: str = ""
    source_path: Optional[Path] = None
```

Cette classe stocke seulement les données utiles d'un fichier `.data` :

- durée de stress journalisée ;
- version de firmware ;
- chemin source.

Il ne faut pas stocker toutes les valeurs simplement parce qu'elles existent dans le fichier. Moins de champs signifie moins de complexité.

6. Fichier `parsers/config_json.py`

Ce fichier lit les configurations JSON de test.

Imports et expressions régulières

```
import json
import re
from pathlib import Path
from typing import Any, Iterator, Optional

from models.data_models import OvenplanEntry, ParsedConfig, TempMode, Zone
```

Rôle :

- `json` lit les fichiers JSON ;
- `re` trouve des motifs dans du texte ;
- `Path` manipule les chemins ;
- les type hints rendent le code plus lisible ;
- les modèles permettent au parseur de renvoyer des objets structurés.

Patterns principaux :

```
_DUT_ID_PATTERN = re.compile(r"(\d+)_(\d+)_(\d+)$")
_ZONE_PATTERN = re.compile(r"(?:^|_)([ABC])(?:_|$)", re.IGNORECASE)
_TEMP_MODE_PATTERN = re.compile(r"\bf_MV\s*=\s*(true|false)\b", re.IGNORECASE)
```

Rôle des patterns :

- `_DUT_ID_PATTERN` reconnaît les noms DUT comme `88_1_2` ;
- `_ZONE_PATTERN` reconnaît A, B ou C dans un nom de test ;
- `_TEMP_MODE_PATTERN` reconnaît `f_MV = true` ou `f_MV = false`.

La regex doit rester précise. Chercher simplement la lettre A trouverait aussi des A dans d'autres mots.

Fonction `_walk_strings`

```
def _walk_strings(value: Any) -> Iterator[str]:
    if isinstance(value, str):
        yield value
    elif isinstance(value, dict):
        for child in value.values():
            yield from _walk_strings(child)
    elif isinstance(value, list):
        for child in value:
            yield from _walk_strings(child)
```

Cette fonction traverse récursivement une structure JSON et renvoie tous les textes trouvés.

Exemple :

```
{
  "Testplans": [
    {"functions": [{"code": "f_MV = true"}]}
  ]
}
```

La fonction trouve :

```
f_MV = true
```

Pourquoi elle existe :

Le réglage `f_MV` peut être profondément imbriqué dans le JSON. La fonction évite de dépendre d'un chemin JSON trop fragile.

Fonction `_parse_zone`

```
def _parse_zone(value: object) -> Optional[Zone]:
    try:
        return Zone(str(value).strip().upper())
    except (TypeError, ValueError):
        return None
```

Cette fonction convertit une valeur en `Zone.A`, `Zone.B` ou `Zone.C`.

Exemples :

```
"a"    -> Zone.A
" A "  -> Zone.A
"D"    -> None
None   -> None
```

Elle échoue proprement au lieu de faire planter le parseur.

Fonction `_zone_from_test_name`

```
def _zone_from_test_name(test_name: str) -> Optional[Zone]:
    match = _ZONE_PATTERN.search(test_name)
    return _parse_zone(match.group(1)) if match else None
```

Cette fonction tente de lire la zone dans le nom du test.

Exemple :

```
run_A_test -> Zone.A
```

Cette valeur sert surtout de fallback si une ligne d'ovenplan ne contient pas la zone.

Fonction `_temp_mode_from_data`

```
def _temp_mode_from_data(data: dict[str, Any]) -> TempMode:
    for text in _walk_strings(data.get("Testplans", [])):
        match = _TEMP_MODE_PATTERN.search(text)
        if match:
            return TempMode.MV if match.group(1).lower() == "true" else TempMode.HV
    return TempMode.HV
```

Règles :

```
f_MV = true -> TempMode.MV
```

```
f_MV = false -> TempMode.HV
absent      -> TempMode.HV
```

Le défaut HV est choisi pour garder un comportement stable quand le champ est absent.

Fonction `parse_config_json`

Début :

```
def parse_config_json(path: str | Path) -> ParsedConfig:
    source = Path(path)
    fallback_name = source.stem
    warnings: list[str] = []
```

Ce bloc normalise le chemin, prépare un nom de test de secours et crée une liste d'avertissements.

Lecture du fichier :

```
try:
    with source.open("r", encoding="utf-8-sig") as handle:
        data = json.load(handle)
except (OSError, UnicodeError, json.JSONDecodeError) as exc:
    return ParsedConfig(
        test_name=fallback_name,
        zone=_zone_from_test_name(fallback_name),
        temp_mode=TempMode.HV,
        source_path=source,
        warnings=[f"JSON non lisible : {exc}"],
    )
```

`utf-8-sig` accepte les fichiers avec marque BOM. Une erreur de fichier ou de JSON renvoie un résultat partiel avec avertissement au lieu d'arrêter tout le chargement du dossier.

Champs principaux :

```
test_name = str(data.get("Test Name") or fallback_name)
fallback_zone = _zone_from_test_name(test_name)
temp_mode = _temp_mode_from_data(data)
instruments = [str(item) for item in data.get("Instruments", []) if item]
entries: list[OvenplanEntry] = []
```

Vérification de l'ovenplan :

```
ovenplan = data.get("Ovenplan", [])
if not isinstance(ovenplan, list):
    ovenplan = []
    warnings.append("Ovenplan n'est pas une liste.")
```

Une donnée JSON externe n'est jamais supposée correcte. Le parseur vérifie les types avant de les utiliser.

Traitement des lignes :

```
for index, row in enumerate(ovenplan, start=1):
    if not isinstance(row, dict):
        warnings.append(f"Entrée ovenplan {index} ignorée : ce n'est pas un objet.")
        continue
```

Une ligne invalide est ignorée, mais les autres lignes restent utilisables.

Lecture des champs DUT, ID et zone :

```
dut_name = str(row.get("DUT") or "").strip()
id_match = _DUT_ID_PATTERN.search(dut_name)
zone = _parse_zone(row.get("Zone")) or fallback_zone
```

Lecture du slot :

```
try:
    position = int(str(row.get("Slot") or "").strip())
except ValueError:
    position = 0
```

Validation :

```
if not dut_name or not zone or position <= 0:
    warnings.append(
        f"Entrée ovenplan {index} ignorée : DUT, zone ou slot invalide."
    )
    continue
```

Construction du résultat :

```
entries.append(
    OvenplanEntry(
        controller_id=int(id_match.group(1)) if id_match else None,
        position=position,
        zone=zone,
        dut_name=dut_name,
        hw_target=str(row.get("HW Target") or "").strip(),
        load_board=str(row.get("LoadBoard") or "").strip(),
        dut_board=str(row.get("DUTBoard") or "").strip(),
        uc_fsm=str(row.get("uC FSM") or "").strip(),
    )
)
```

Si le DUT s'appelle aa au lieu de 88_1_2, controller_id vaut None. C'est volontaire : les noms DUT libres sont valides.

Résultat final :

```
return ParsedConfig(
    test_name=test_name,
    zone=fallback_zone or (entries[0].zone if entries else None),
    temp_mode=temp_mode,
    instruments=instruments,
    ovenplan_entries=entries,
    source_path=source,
    warnings=warnings,
)
```

7. Fichier parsers/mtpx.py

Ce fichier lit la durée de test planifiée dans un fichier .mtpx.

Imports et opérateurs autorisés

```
import ast
import json
import operator
import re
from pathlib import Path
from typing import Any, Iterator, Optional
```

Pourquoi ces bibliothèques :

- json charge le fichier ;
- ast lit une expression mathématique de manière contrôlée ;
- operator fournit les vraies fonctions de calcul ;

- `re` sert à la recherche fallback dans du texte ;
- `Path` gère les chemins.

Opérateurs autorisés :

```
_BINARY_OPERATORS = {
    ast.Add: operator.add,
    ast.Sub: operator.sub,
    ast.Mult: operator.mul,
    ast.Div: operator.truediv,
}
_UNARY_OPERATORS = {ast.UAdd: operator.pos, ast.USub: operator.neg}
```

Le parseur autorise seulement :

`+ - * /`

Pourquoi ne pas utiliser `eval()` ?

`eval()` peut exécuter du code Python arbitraire. C'est dangereux avec des contenus de fichiers.

À éviter :

```
eval("__import__('os').system('echo no')")
```

Meilleure solution :

Lire l'expression avec `ast`, puis accepter seulement les noeuds nécessaires aux calculs.

Pattern `_STOP_TIME_PATTERN`

```
_STOP_TIME_PATTERN = re.compile(
    r"\bstop\s*=\s*\{.*?\btime\s*=\s*([0-9eE+-*/().\s]+)",
    re.DOTALL,
)
```

Ce pattern trouve du texte comme :

```
stop = { time = 1001*3600 }
```

Il limite la capture aux caractères utiles pour les nombres et les calculs.

Fonction `_safe_number`

```
def _safe_number(expression: str) -> Optional[float]:
    try:
        tree = ast.parse(expression.strip(), mode="eval")
    except SyntaxError:
        return None
```

La fonction transforme un texte en arbre syntaxique. Une syntaxe invalide donne `None`.

Évaluation :

```
def evaluate(node: ast.AST) -> float:
    if isinstance(node, ast.Expression):
        return evaluate(node.body)
    if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
        return float(node.value)
    if isinstance(node, ast.BinOp) and type(node.op) in _BINARY_OPERATORS:
        return _BINARY_OPERATORS[type(node.op)](
            evaluate(node.left), evaluate(node.right)
        )
```

```

if isinstance(node, ast.UnaryOp) and type(node.op) in _UNARY_OPERATORS:
    return _UNARY_OPERATORS[type(node.op)](evaluate(node.operand))
raise ValueError("Unsupported expression")

```

Exemple accepté :

```
1001*3600
```

Exemple refusé :

```
__import__('os').system('echo no')
```

Fin :

```

try:
    value = evaluate(tree)
except (ValueError, TypeError, ZeroDivisionError, OverflowError):
    return None
return value if value >= 0 else None

```

Une durée planifiée négative n'a pas de sens métier, donc elle est refusée.

Fonction `_walk`

```

def _walk(value: Any) -> Iterator[Any]:
    yield value
    if isinstance(value, dict):
        for child in value.values():
            yield from _walk(child)
    elif isinstance(value, list):
        for child in value:
            yield from _walk(child)

```

Cette fonction parcourt récursivement toutes les valeurs JSON.

Pourquoi elle existe :

Le champ `stop_time` peut se trouver à différents endroits dans un fichier MTPX.

Fonction `parse_planned_test_seconds`

```

def parse_planned_test_seconds(path: str | Path) -> Optional[float]:
    source = Path(path)
    try:
        with source.open("r", encoding="utf-8-sig") as handle:
            data = json.load(handle)
    except (OSError, UnicodeError, json.JSONDecodeError):
        return None

```

La fonction lit le fichier MTPX comme JSON. Si la lecture échoue, elle retourne `None`.

Recherche structurée :

```

for item in _walk(data):
    if not isinstance(item, dict):
        continue
    if str(item.get("templateName", "")).strip() != "stop_time":
        continue
    value = _safe_number(str(item.get("templateValue", "")))
    if value is not None:
        return value

```

Elle cherche une structure comme :

```
{
```

```

    "templateName": "stop_time",
    "templateValue": "1001*3600"
}

```

Recherche fallback :

```

for item in _walk(data):
    if not isinstance(item, str):
        continue
    match = _STOP_TIME_PATTERN.search(item)
    if match:
        value = _safe_number(match.group(1))
        if value is not None:
            return value
return None

```

Le parseur essaie d'abord la structure propre, puis le texte libre.

8. Fichier `parsers/board_data.py`

Ce fichier lit les fichiers `.data`.

Imports

```

import json
from pathlib import Path
from typing import Optional

from models.data_models import BoardMetadata

```

Fonction `parse_board_data`

```

def parse_board_data(path: str | Path) -> Optional[BoardMetadata]:
    source = Path(path)

```

Le chemin est normalisé en objet `Path`.

Lecture :

```

try:
    with source.open("r", encoding="utf-8-sig") as handle:
        data = json.load(handle)
except (OSError, UnicodeError, json.JSONDecodeError):
    return None

```

Une DATA absente ou cassée ne doit pas arrêter toute l'analyse d'un dossier.

Lecture sûre de la structure :

```

test_info = data.get("Test Info") or {}
history = data.get("HW History") or []
latest = history[-1] if isinstance(history, list) and history else {}
hardware = latest.get("HW Info") or {}
version = hardware.get("version") or {}

```

Pourquoi `history[-1]` ?

Le dernier élément de `HW History` est considéré comme le plus récent.

À éviter :

```

version = data["HW History"][-1]["HW Info"]["version"]

```

Ce code plante dès qu'un champ manque.

Lecture des secondes :

```
try:
    seconds = float(test_info.get("Seconds") or 0.0)
except (TypeError, ValueError):
    seconds = 0.0
```

Résultat :

```
return BoardMetadata(
    log_stress_seconds=max(0.0, seconds),
    firmware_version=str(version.get("fw") or ""),
    source_path=source,
)
```

`max(0.0, seconds)` empêche une durée de stress négative.

9. Tests

Les tests décrivent le comportement attendu du code.

Exemples de cas testés :

- un JSON avec ovenplan et mode MV ;
- un DUT nommé librement, par exemple aa ;
- un fichier JSON cassé ;
- une expression MTPX sûre comme `1001*3600` ;
- une expression MTPX dangereuse à refuser ;
- la durée de stress et la version firmware depuis DATA ;
- les dossiers contenant plusieurs campagnes de test ;
- les zones multiples et jusqu'à 24 boards.

Commande :

```
pytest -q
```

Pourquoi écrire des petits tests synthétiques ?

Ils sont rapides, précis et faciles à comprendre. Tester seulement avec de gros fichiers réels rend les erreurs plus difficiles à isoler.

10. Fichier `.gitignore`

Extrait :

```
/* .tdms
/* .tdms_index
/* .log
/* .data
/* .store
/* .mtpx
/* degree.json
```

Rôle :

Les données brutes ne doivent pas être commitées dans Git.

Pourquoi ?

Les dossiers de mesure DHTOL peuvent être très volumineux. Ils peuvent aussi contenir des données internes ou sensibles.

Bonne pratique :

Si des exemples sont nécessaires, créer de petites fixtures anonymisées dans `tests/fixtures/`.

11. Fichier `requirements.txt`

Exemple minimal :

```
pytest>=8.0
```

Ce fichier liste les dépendances Python nécessaires.

Principe :

Ne pas ajouter trop tôt toutes les bibliothèques prévues pour les étapes futures. Chaque dépendance doit avoir une raison actuelle.

12. Pourquoi le code est écrit ainsi ?

Principe de base :

Lire le fichier brut -> créer une structure propre -> vérifier avec des tests

Avantages :

- les parseurs restent petits ;
- les modèles restent lisibles ;
- les tests tournent vite ;
- les responsabilités sont séparées ;
- chaque étape peut être expliquée et corrigée seule.

Bon engineering :

Ne pas garder du code futur qui n'est pas encore utilisé. Construire une couche fiable, la tester, puis passer à la couche suivante.

13. Alternatives

Alternative 1 : seulement des dictionnaires

Avantages :

- rapide à écrire ;
- moins de classes.

Inconvénients :

- fautes de frappe dans les clés ;
- lecture plus difficile ;
- refactoring plus fragile.

Solution choisie :

`Dataclasses`

Alternative 2 : Pydantic

Avantages :

- validation plus forte ;
- bons messages d'erreur.

Inconvénients :

- dépendance supplémentaire ;
- apprentissage plus lourd pour un projet junior.

Solution choisie :

`Bibliothèque standard + dataclasses`

Alternative 3 : exceptions pour chaque fichier cassé

Avantages :

- les erreurs sont visibles immédiatement.

Inconvénients :

- un seul fichier cassé peut arrêter l'analyse complète du dossier.

Solution choisie :

`None` ou avertissements pour les problèmes attendus

Alternative 4 : `eval()` pour les expressions MTPX

Avantages :

- très court.

Inconvénients :

- dangereux ;
- exécute potentiellement du code arbitraire.

Solution choisie :

ast avec opérateurs autorisés

14. Erreurs de débutant à éviter

1. Tout écrire dans une seule grande fonction.

Mieux :

```
models/      -> formes de données
parsers/     -> lecture de fichiers
analysis/    -> logique métier
visualization/ -> affichage
tests/       -> comportement attendu
```

2. Stocker trop de champs.

Garder seulement ce qui est utilisé maintenant.

3. Utiliser `eval()` sur le contenu de fichiers.

Ne jamais faire cela. Utiliser un parseur sûr.

4. Faire confiance aveuglément à une structure JSON.

Utiliser `.get()`, vérifier les types et gérer les erreurs attendues.

5. Utiliser une liste comme valeur par défaut directe.

Mauvais :

```
warnings: list[str] = []
```

Bon :

```
warnings: list[str] = field(default_factory=list)
```

6. Confondre controller ID et slot.

controller_id vient du nom DUT
position vient du champ Slot

7. Commiter les données brutes.

Utiliser `.gitignore`.

15. Comment construire ce type de projet depuis zéro

Étape 1 : définir un objectif petit.

Lire un fichier JSON.
Créer une liste de boards.
Écrire un test.

Étape 2 : créer les dossiers.

```
models/
parsers/
```

```
tests/  
docs/
```

Étape 3 : écrire le premier modèle.

```
@dataclass  
class OvenplanEntry:  
    position: int  
    dut_name: str
```

Étape 4 : écrire le parseur.

```
def parse_config_json(path: str | Path) -> ParsedConfig:  
    ...
```

Étape 5 : créer une mini-fichier de test dans le test.

```
source.write_text(json.dumps({"Ovenplan": [...]}))
```

Étape 6 : lancer les tests.

```
pytest -q
```

Étape 7 : ajouter le parseur suivant.

Ordre recommandé :

```
JSON -> MTPX -> DATA -> LOG -> TDMS -> analyse -> visualisation
```

Étape 8 : documenter.

Écrire pourquoi les champs existent, pourquoi certains champs sont ignorés et quels comportements sont garantis par les tests.

16. Résumé d'apprentissage

À retenir :

- les modèles définissent la forme des données propres ;
- les parseurs transforment des fichiers irréguliers en objets fiables ;
- les tests prouvent le comportement ;
- les enums évitent les chaînes libres ambiguës ;
- les dataclasses conviennent bien aux petits objets de données ;
- `field(default_factory=list)` évite les listes partagées ;
- `ast` est plus sûr que `eval()` ;
- `.gitignore` protège le repository contre les données brutes volumineuses ;
- un code simple et testé vaut mieux qu'une architecture future inutilisée.

Phrase clé :

Ne construis pas tout l'analyser en une seule fois.
Construis une couche fiable, teste-la, documente-la, puis passe à la suivante.